



University of
Nottingham

UK | CHINA | MALAYSIA

Lecture 8: Artificial Neural Networks

COMP1037 Fundamentals of Artificial Intelligence

Xinan Chen

University of Nottingham Ningbo China

April 2025

Outline |

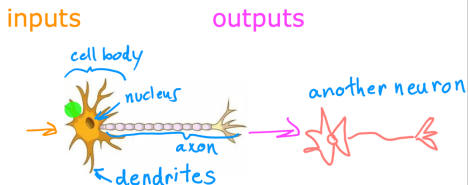
1. Architecture of Neural Networks
2. Perceptron
3. Training a Neural Network
4. Evaluating a Neural Network

Neural Networks |

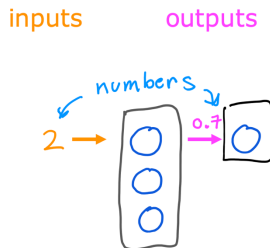
- Origins: Algorithms that try to mimic the brain.
- Used in the 1980's and early 1990's.
- Fell out of favor in the late 1990's.
- Resurgence from around 2005.
- Speech $\rightarrow$ images $\rightarrow$ text (NLP) $\Rightarrow$ Matrix & Vector

Neurons in the Brain |

Biological neuron

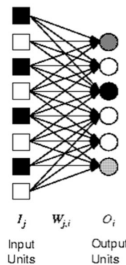
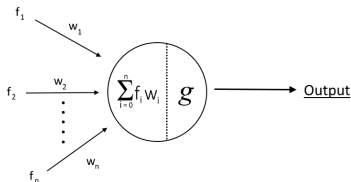


Simplified mathematical model of a neuron

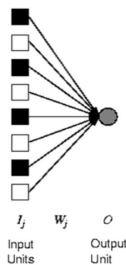


Perceptron: Single Layer NN |

- **Input layer:** the item features.
- **Output layer:** Each node in the output layer has n synapses (for every item feature), each with its own weight. The nodes find the dot product of the item features and the synapse weights. These values then pass through an **activation function**. Finally, we pick the largest of the values and return its index.



Perceptron Network



Single Perceptron

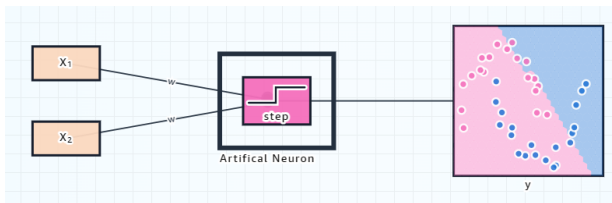
Activation Functions & Artificial Neurons |

In a neural network, this function node is very special – we call it an **artificial neuron**.

An artificial neuron is a fundamental computational element that:

- Receives inputs
- Performs a weighted operation on these inputs
- Passes the result through a function

In neural networks, these functions must be **non-linear**, and are referred to as **activation functions**.



Activation Functions |

Activation functions are at the heart of artificial neurons in a neural network.

- These crucial components introduce **non-linearity** into the model.
- They transform the weighted inputs to generate an output.
- The non-linear nature is essential for neural networks to learn from complex data.

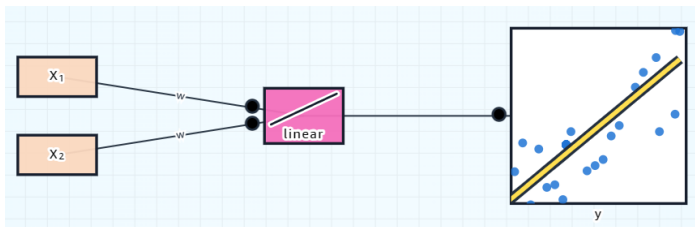
Activation Functions |

Name	Plot	Function	Description
Sigmoid (logistic)		$f(x) = \frac{1}{1 + e^{-x}}$	Squashes input to (0, 1).
Hyperbolic Tangent (tanh)		$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Squashes input to (-1, 1).
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$	Only keep positive values.
Step Function (Perceptron) ^[1]		$f(x) = \begin{cases} +1, & \text{if } x \geq 0 \\ -1, & \text{if } x < 0 \end{cases}$	Returns only -1 or 1 (neuron fires or doesn't fire).

Linear Regression |

Represent Linear Regression as a computational graph:

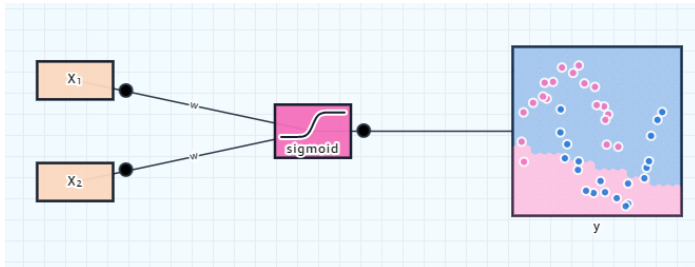
1. Add weights to each edge connecting the input nodes to the function nodes.
2. Update our function to represent a linear function of a weighted sum.



Logistic Regression |

Represent Logistic Regression as a computational graph:

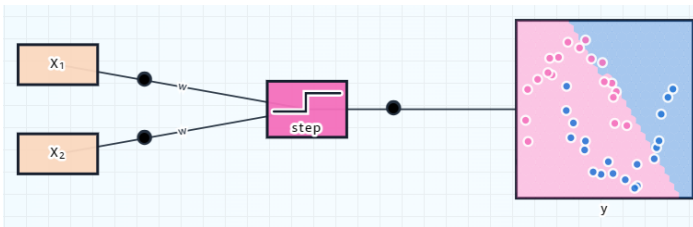
1. Change the linear function to a **sigmoid function**.



Perceptron |

Represent Perceptron as a computational graph:

1. Switch logistic function to a **step function**.

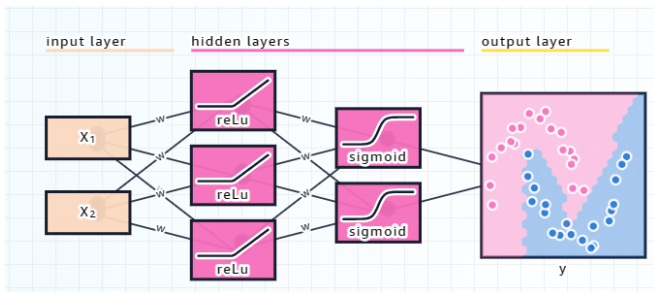


Neural Network Architecture |

A neural network architecture consists of three layer types:

- **Input layer:** A layer with a node for each network input.
- **Hidden layer(s):** A layer full of artificial neurons.
- **Output layer:** A layer representing the network's output.

There should only be one input and one output layer, but there may be an arbitrary number of hidden layers.



Multiple Hidden Layers |

Effect of hidden layers:

- **Wide:** having many neurons in a given hidden layer.
- **Deep:** having many hidden layers in the network.
- Each layer adds degree of non-linearity.
- Balancing neuron count optimizes performance; more neurons enable complex learning, at the cost of the risk of overfitting and more computational cost.



Deep Learning |

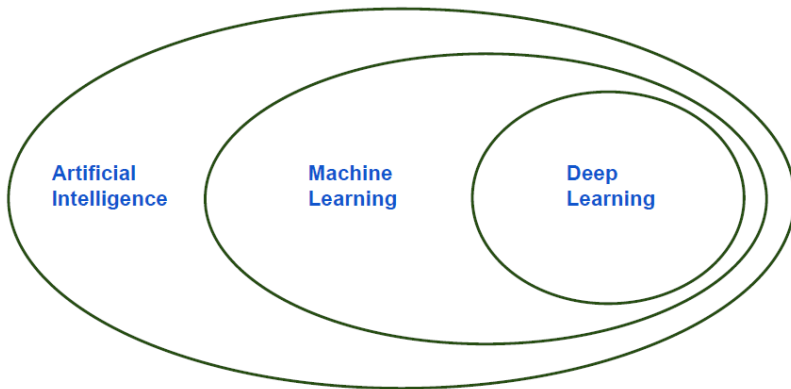
Deep learning itself is **not new**:

- Neural networks have been around since the 70s.
- Deep neural networks (multiple hidden layers) are not much younger.

Why everybody is talking about deep learning **now**:

1. Powerful hardware allows training of **huge neural networks**.
2. Large amount of data is available.
3. Special network architectures for image/text data.
4. Better optimization and regularization strategies.

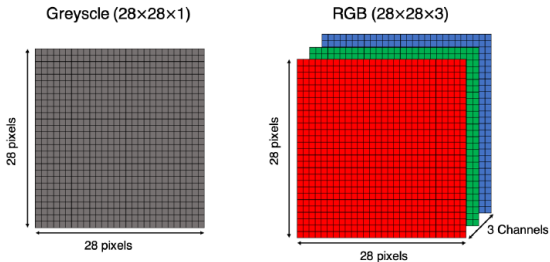
Deep Learning |



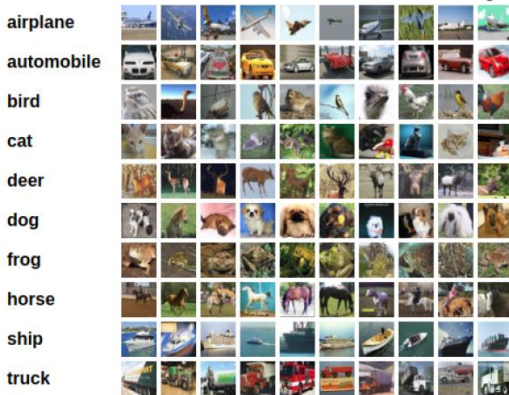
Possible Use-case: Images |

- **High Dimensional:** A color image with 255×255 (3 Colors) pixels already has 195075 features.
- **Informative:** A single pixel is not meaningful in itself.
- **Training Data:** Depending on applications huge amounts of data are available.

Architecture: **Convolutional Neural Networks (CNN)**



Possible Use-case: Images (cont.) |

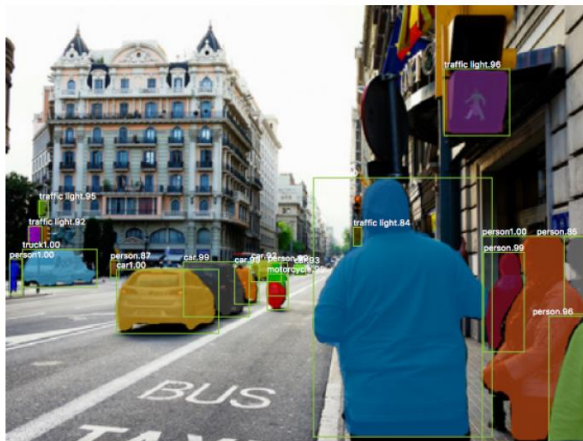


Credit: Alex Krizhevsky (2009)

Image classification tries to predict a single label for each image.

CIFAR-10 is a well-known dataset used for image classification. It consists of 60,000 32x32 color images containing one of 10 object classes, with 6000 images per class.

Possible Use-case: Images (cont.) |



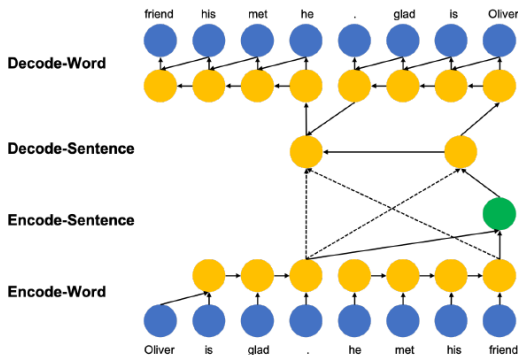
Credit: Kaiming He (2017)

Object Detection Mask R-CNN is a general framework for instance segmentation, that efficiently detects objects in an image while simultaneously generating a high-quality segmentation mask for each instance.

Possible Use-case: Text |

- **High Dimensional:** Each word can be a single feature (300000 words in the German language).
- **Informative:** A single word does not provide much context.
- **Training Data:** Huge amounts of text data available.

Architecture: **R**ecurrent **N**eural **N**etworks (RNN)



Possible Use-case: Text (cont.) |

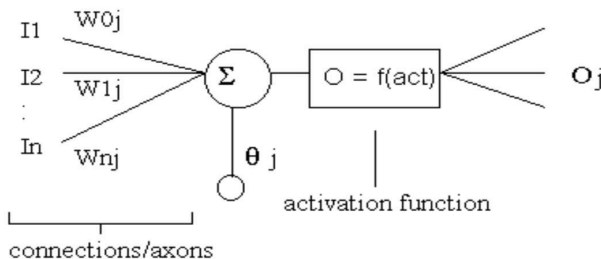


Machine Translation (e.g. google translate) Neural machine translation exploits neural networks to predict the likelihood of a sequence of words, typically modeling entire sentences in a single integrated model.

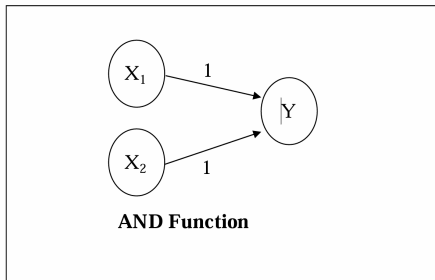
The First Neural Networks |

Using neural networks, we can model **logic functions**.

Let's look at 4 logic functions:



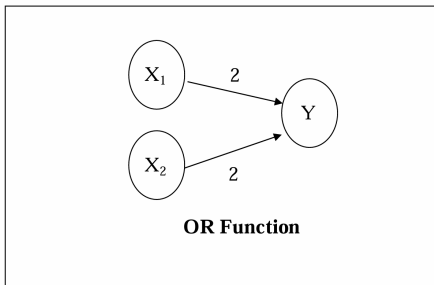
The First Neural Networks – AND |



AND		
X1	X2	Y
1	1	1
1	0	0
0	1	0
0	0	0

$$\text{Threshold}(Y) = 2$$

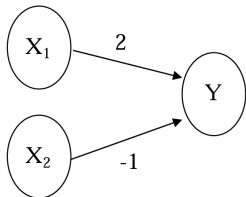
The First Neural Networks – OR |



$$\text{Threshold}(Y) = 2$$

OR		
X1	X2	Y
1	1	1
1	0	1
0	1	1
0	0	0

The First Neural Networks – AND NOT |



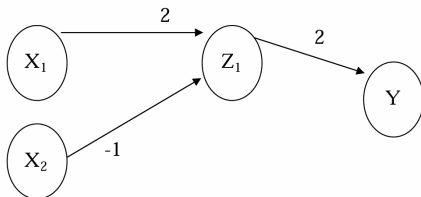
AND NOT Function

AND NOT		
X1	X2	Y
1	1	0
1	0	1
0	1	0
0	0	0

$$\text{Threshold}(Y) = 2$$

The First Neural Networks – XOR (cont.) |

$$X_1 \text{ XOR } X_2 = (X_1 \text{ AND NOT } X_2) \text{ OR } (X_2 \text{ AND NOT } X_1)$$

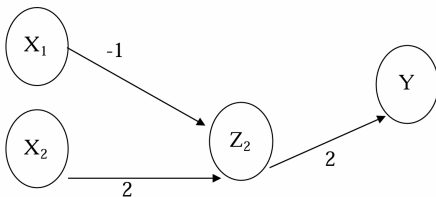


XOR		
X1	X2	Y
1	1	0
1	0	1
0	1	1
0	0	0

AND NOT		
X1	X2	Y
1	1	0
1	0	1
0	1	0
0	0	0

The First Neural Networks – XOR |

$$X_1 \text{ XOR } X_2 = (X_1 \text{ AND NOT } X_2) \text{ OR } (X_2 \text{ AND NOT } X_1)$$

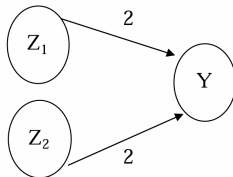


XOR		
X1	X2	Y
1	1	0
1	0	1
0	1	1
0	0	0

AND NOT		
X2	X1	Y
1	1	0
1	0	1
0	1	0
0	0	0

The First Neural Networks – XOR (cont.) |

$$X_1 \text{ XOR } X_2 = (X_1 \text{ AND NOT } X_2) \text{ OR } (X_2 \text{ AND NOT } X_1)$$

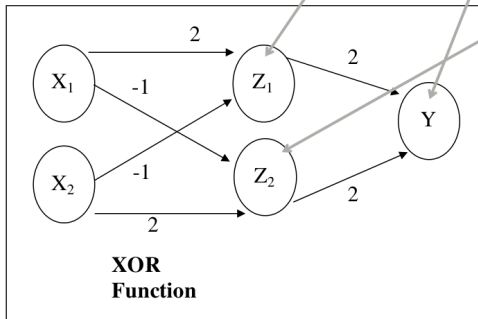


XOR		
X1	X2	Y
1	1	0
1	0	1
0	1	1
0	0	0

AND NOT		
X1	X2	Y
1	1	0
1	0	1
0	1	0
0	0	0

The First Neural Networks – XOR Function |

$$X_1 \text{ XOR } X_2 = (X_1 \text{ AND NOT } X_2) \text{ OR } (X_2 \text{ AND NOT } X_1)$$



XOR		
X1	X2	Y
1	1	0
1	0	1
0	1	1
0	0	0

The First Neural Networks – XOR Truth Table |

$$X_1 \text{ XOR } X_2 = (X_1 \text{ AND NOT } X_2) \text{ OR } (X_2 \text{ AND NOT } X_1)$$

XOR	
X1	X2
1	1
1	0
0	1
0	0

Y
0
1
1
0

Linear Separability |

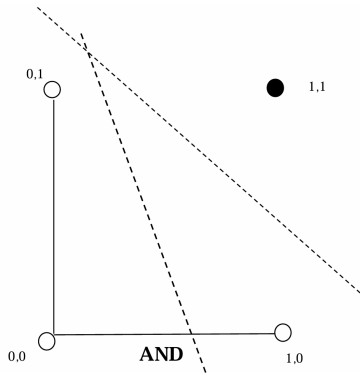
- Functions which can be separated by a straight line are called **Linearly Separable**.
- Only linearly separable functions can be represented by a **single layer NN** (perceptron).

Input 1

Input 2

Output

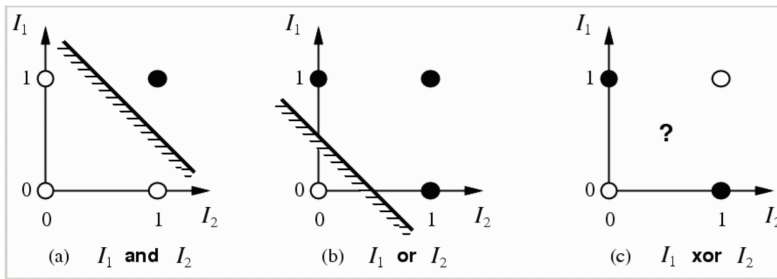
AND			
0	0	1	1
0	1	0	1
0	0	0	1



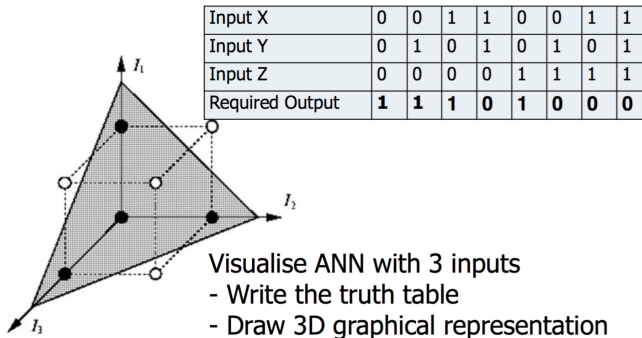
Linear Separability (cont.) |

Linear Separability is also possible in more than 3 dimensions – but it is harder to visualize.

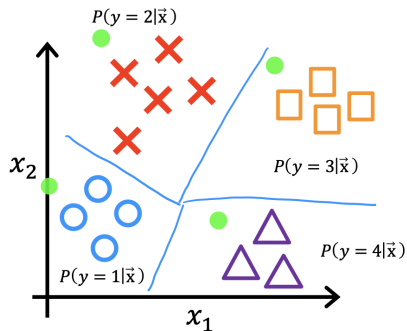
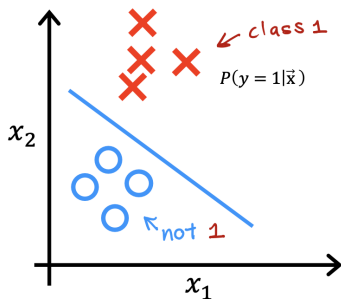
Minsky & Papert



Linear Separability – XOR is Not Linearly Separable |



Multiclass Classification |



Multiclass Classification: Softmax Regression |

Logistic regression
(2 possible output values)

$$z = \vec{w} \cdot \vec{x} + b$$

$$\times a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1|\vec{x}) \quad 0.11$$

$$\circ a_2 = 1 - a_1 = P(y=0|\vec{x}) \quad 0.29$$

Softmax regression
(N possible outputs) $y=1,2,3,\dots,N$

$$z_j = \vec{w}_j \cdot \vec{x} + b_j \quad j = 1, \dots, N$$

$$a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y=j|\vec{x})$$

Softmax regression (4 possible outputs) $y=1,2,3,4$

$$\times z_1 = \vec{w}_1 \cdot \vec{x} + b_1$$

$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=1|\vec{x}) \quad 0.30$$

$$\circ z_2 = \vec{w}_2 \cdot \vec{x} + b_2$$

$$a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=2|\vec{x}) \quad 0.20$$

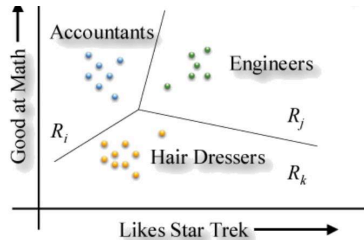
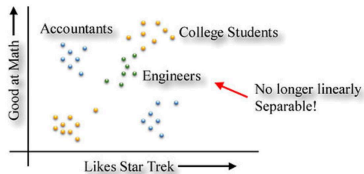
$$\square z_3 = \vec{w}_3 \cdot \vec{x} + b_3$$

$$a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=3|\vec{x}) \quad 0.15$$

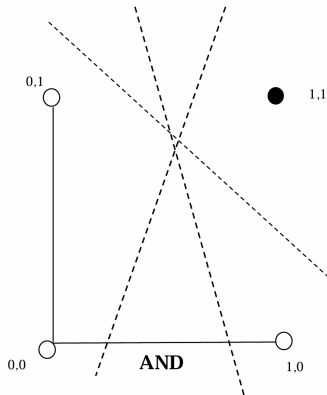
$$\triangle z_4 = \vec{w}_4 \cdot \vec{x} + b_4$$

$$a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}} \\ = P(y=4|\vec{x}) \quad 0.35$$

Linear Separability – Multi-class |



Training a Neural Network |

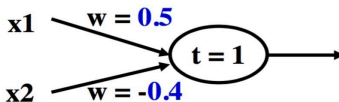


Input 1
Input 2
Output

AND			
0	0	1	1
0	1	0	1
0	0	0	1

Training a Neural Network (cont.) |

Training a NN

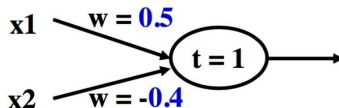


AND		
X1	X2	T
0	0	0
0	1	0
1	0	0
1	1	1

X1	X2	Summation	Output
0	0	$(0 * 0.5) + (0 * -0.4) = 0$	0
0	1		
1	0		
1	1		

Training a Neural Network (cont.) |

Training a NN



AND			
X1	X2	T	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0

X1	X2	Summation	Output
0	0	$(0 * 0.5) + (0 * -0.4) = 0$	0
0	1	$(0 * 0.5) + (1 * -0.4) = -0.4$	0
1	0	$(1 * 0.5) + (0 * -0.4) = 0.5$	0
1	1	$(1 * 0.5) + (1 * -0.4) = 0.1$	0

Training a Neural Network (cont.) |

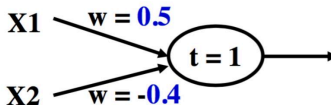
Training a NN

While epoch produces an error

Check next inputs (pattern) from epoch

AND			
X1	X2	T	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0

End While



Epoch: The entire training set feed into the neural network.
 The AND function: an epoch consists of four sets of inputs (patterns) feed into the network ([0,0], [0,1], [1,0], [1,1])

Training a Neural Network (cont.) |

Training a NN

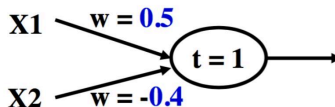
While epoch produces an error

Check next inputs (pattern) from epoch

$$\text{Err} = T - O$$

AND			
X1	X2	T	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0

End While



Training Value, T : The value that we require the network to produce. For example, with [0,0] for the AND function the training value will be 0.

Training a Neural Network (cont.) |

Training a NN

While epoch produces an error

Check next inputs (pattern) from epoch

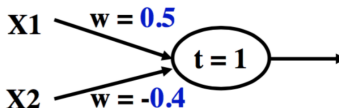
$$\text{Err} = T - O$$

If $\text{Err} \neq 0$ then

End If

End While

AND			
X1	X2	T	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0



Error, Err : The amount the output by the network O differs from the training value T.

Training a Neural Network (cont.) |

Training a NN

What is the problem if the learning rate is set too high, or too low?

While epoch produces an error

Check next inputs (pattern) from epoch

$$\text{Err} = T - O$$

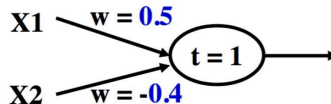
If $\text{Err} \neq 0$ then

$$w_i = w_i + \text{LR} * X_i * \text{Err}$$

End If

End While

AND			
X1	X2	T	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0



X_i : Inputs to the neuron

w_i : Weight from input X_i to the output

LR: The learning rate: how quickly the network converges.
It is set by experimentation, typically 0.1

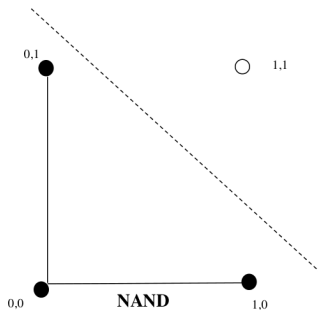
Training a Perceptron |

Can we train a perceptron to learn logical NAND?

Training neural networks isn't easy! Even for small, relatively simple datasets such as those shown here, training a neural network can take some care and finesse.

Input 1
Input 2
Output

NAND			
0	0	1	1
0	1	0	1
1	1	1	0



The Learning Process |

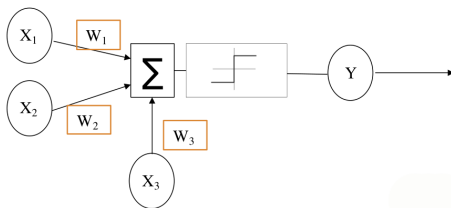
Algorithm

Randomly assign initial weights

Do:

- Present the network with input
- Calculate the error value in the output
- Adjust the weightings of the inputs according to the error

Until no error value exists for all inputs



Training a Perceptron – Setup |

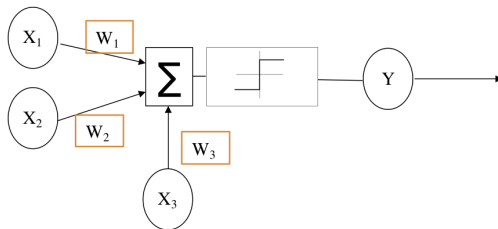
NAND Truth Table

Input 1	Input 2	Output
0	0	1
0	1	1
1	0	1
1	1	0

Parameters

- Threshold = 0.6
- Learning rate = 0.1

Note: X_3 is constant 1 for bias

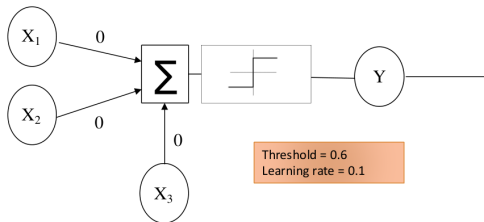


Perceptron Training – Epoch 1, Step 1 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.0	0.0			
0	1	1							
1	0	1							
1	1	1							



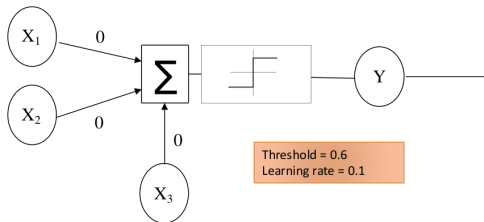
Threshold = 0.6
Learning rate = 0.1

Perceptron Training – Epoch 1, Step 1 (Error) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.0	0.0	0	1	+0.1
0	1	1							
1	0	1							
1	1	1							

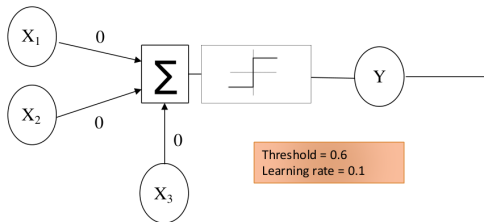


Perceptron Training – Epoch 1, Step 2 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.0	0.0	0	1	+0.1
0	1	1	0.0	0.0	0.1	0.1			
1	0	1							
1	1	1							

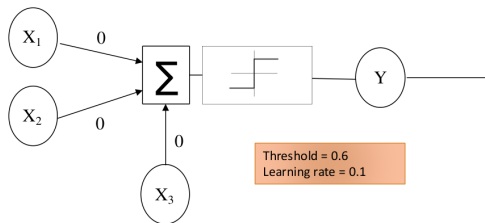


Perceptron Training – Epoch 1, Step 2 (Error) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.0	0.0	0	1	+0.1
0	1	1	0.0	0.0	0.1	0.1	0	1	+0.1
1	0	1							
1	1	1							



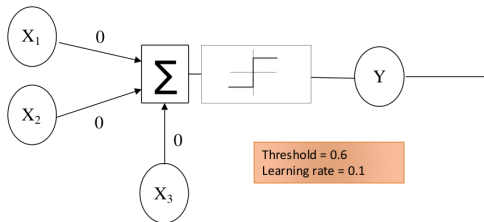
Threshold = 0.6
Learning rate = 0.1

Perceptron Training – Epoch 1, Step 3 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.0	0.0	0	1	+0.1
0	1	1	0.0	0.0	0.1	0.1	0	1	+0.1
→ 1	0	1	0.0	0.1	0.2				
1	1	1							

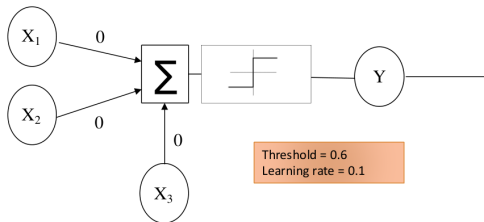


Perceptron Training – Epoch 1, Step 3 (Error) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.0	0.0	0	1	+0.1
0	1	1	0.0	0.0	0.1	0.1	0	1	+0.1
→ 1	0	1	0.0	0.1	0.2	0.2	0	1	+0.1
1	1	1							

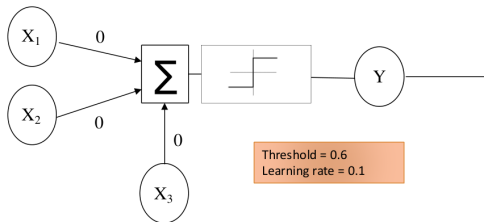


Perceptron Training – Epoch 1, Step 4 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.0	0.0	0	1	+0.1
0	1	1	0.0	0.0	0.1	0.1	0	1	+0.1
1	0	1	0.0	0.1	0.2	0.2	0	1	+0.1
1	1	1	0.1	0.1	0.3	0.5			

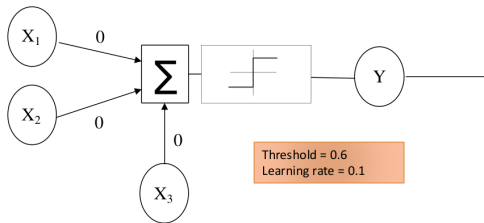


Perceptron Training – Epoch 1, Step 4 (No Error) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.0	0.0	0	1	+0.1
0	1	1	0.0	0.0	0.1	0.1	0	1	+0.1
1	0	1	0.0	0.1	0.2	0.2	0	1	+0.1
1	1	1	0.1	0.1	0.3	0.5	0	0	∓ 0.0

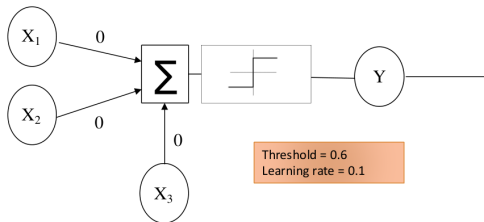


Perceptron Training – Epoch 2 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.1	0.1	0.3	0.3			
0	1	1							
1	0	1							
1	1	1							

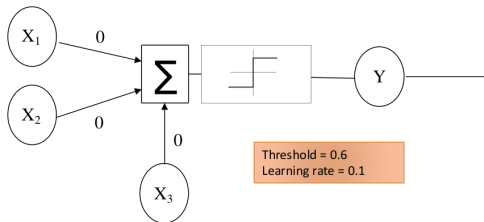


Perceptron Training – Epoch 2 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.1	0.1	0.3	0.3	0	1	+0.1
0	1	1							
1	0	1							
1	1	1							

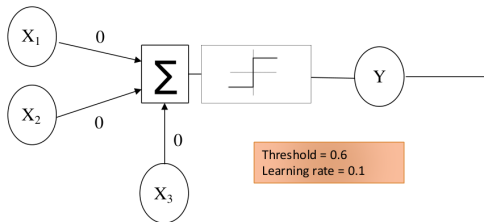


Perceptron Training – Epoch 2 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.1	0.1	0.3	0.3	0	1	+0.1
0	1	1	0.1	0.1	0.4	0.5			
1	0	1							
1	1	1							

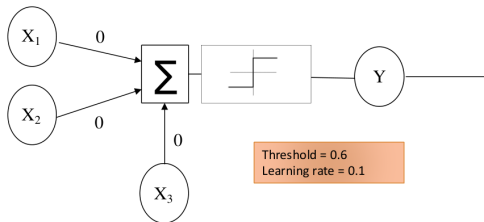


Perceptron Training – Epoch 2 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.1	0.1	0.3	0.3	0	1	+0.1
0	1	1	0.1	0.1	0.4	0.5	0	1	+0.1
1	0	1							
1	1	1							

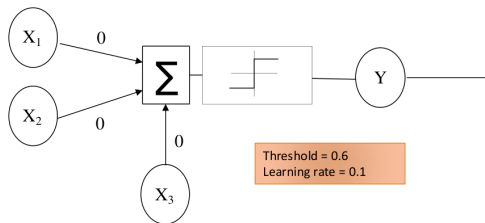


Perceptron Training – Epoch 2 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.1	0.1	0.3	0.3	0	1	+0.1
0	1	1	0.1	0.1	0.4	0.5	0	1	+0.1
→ 1	0	1	0.1	0.2	0.5	0.6			
1	1	1							

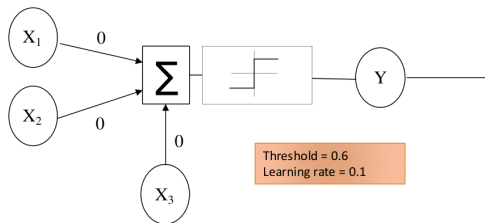


Perceptron Training – Epoch 2 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.1	0.1	0.3	0.3	0	1	+0.1
0	1	1	0.1	0.1	0.4	0.5	0	1	+0.1
→ 1	0	1	0.1	0.2	0.5	0.6	1	0	-0.0
1	1	1							

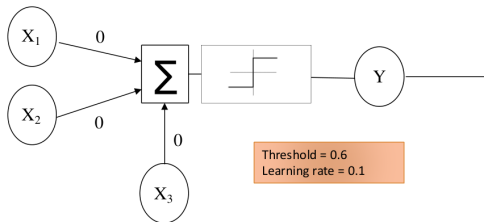


Perceptron Training – Epoch 2 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.1	0.1	0.3	0.3	0	1	+0.1
0	1	1	0.1	0.1	0.4	0.5	0	1	+0.1
1	0	1	0.1	0.2	0.5	0.6	1	0	-0.0
1	1	1	0.1	0.2	0.5	0.8			

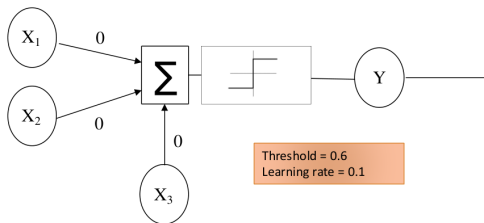


Perceptron Training – Epoch 2 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.1	0.1	0.3	0.3	0	1	+0.1
0	1	1	0.1	0.1	0.4	0.5	0	1	+0.1
1	0	1	0.1	0.2	0.5	0.6	1	0	∓ 0.0
1	1	1	0.1	0.2	0.5	0.8	1	-1	-0.1

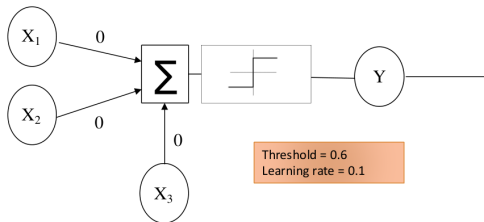


Perceptron Training – Epoch 3 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.1	0.4				
0	1	1							
1	0	1							
1	1	1							

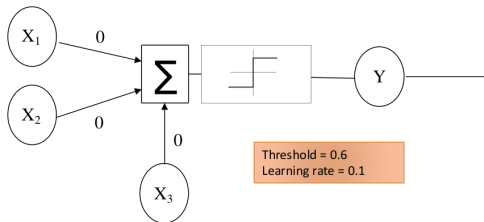


Perceptron Training – Epoch 3 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.1	0.4	0.4	0	1	+0.1
0	1	1							
1	0	1							
1	1	1							



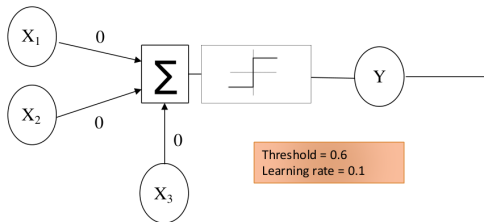
Threshold = 0.6
Learning rate = 0.1

Perceptron Training – Epoch 3 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.1	0.4	0.4	0	1	+0.1
0	1	1	0.0	0.1	0.5				
1	0	1							
1	1	1							

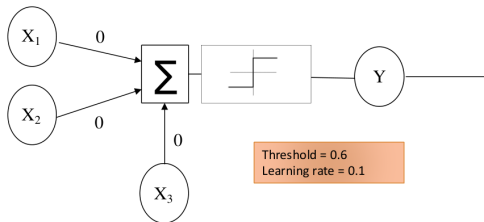


Perceptron Training – Epoch 3 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.1	0.4	0.4	0	1	+0.1
0	1	1	0.0	0.1	0.5	0.6	1	0	-0.0
1	0	1							
1	1	1							

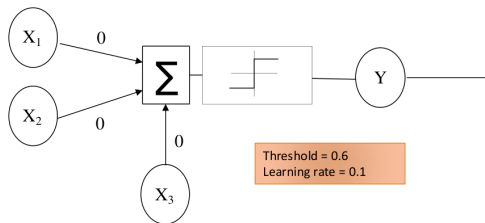


Perceptron Training – Epoch 3 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.1	0.4	0.4	0	1	+0.1
0	1	1	0.0	0.1	0.5	0.6	1	0	-0.0
→ 1	0	1	0.0	0.1	0.5	0.5	0	1	+0.1
1	1	1							

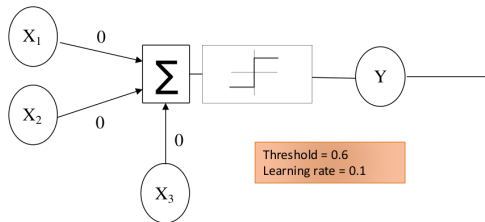


Perceptron Training – Epoch 3 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.1	0.4	0.4	0	1	+0.1
0	1	1	0.0	0.1	0.5	0.6	1	0	-0.0
1	0	1	0.0	0.1	0.5	0.5	0	1	+0.1
1	1	1	0.1	0.1	0.6	0.8	1	-1	-0.1

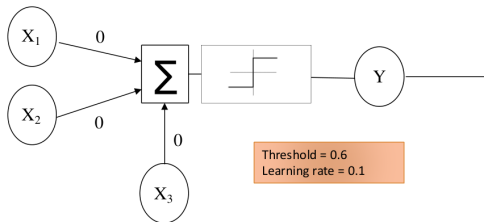


Perceptron Training – Epoch 4 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.5	0.5	0	1	+0.1
0	1	1							
1	0	1							
1	1	1							

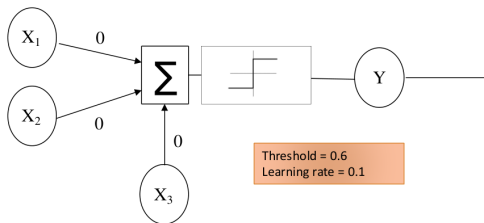


Perceptron Training – Epoch 4 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.5	0.5	0	1	+0.1
0	1	1	0.0	0.0	0.6	0.6	1	0	-0.0
1	0	1							
1	1	1							

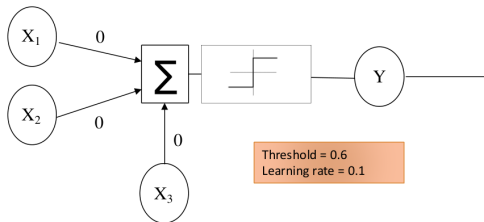


Perceptron Training – Epoch 4 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.5	0.5	0	1	+0.1
0	1	1	0.0	0.0	0.6	0.6	1	0	∓ 0.0
→ 1	0	1	0.0	0.0	0.6	0.6	1	0	∓ 0.0
1	1	1							

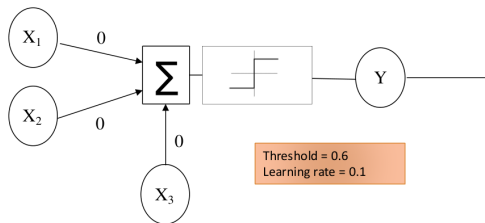


Perceptron Training – Epoch 4 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	0.0	0.0	0.5	0.5	0	1	+0.1
0	1	1	0.0	0.0	0.6	0.6	1	0	∓ 0.0
1	0	1	0.0	0.0	0.6	0.6	1	0	∓ 0.0
1	1	1	0.0	0.0	0.6	0.6	1	-1	-0.1

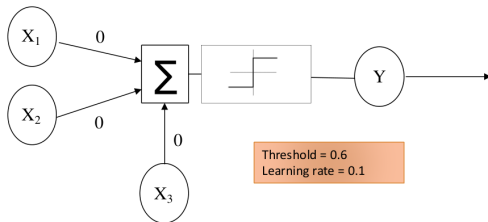


Perceptron Training – Epoch 5 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.1	-0.1	0.5	0.5	0	1	+0.1
0	1	1							
1	0	1							
1	1	1							

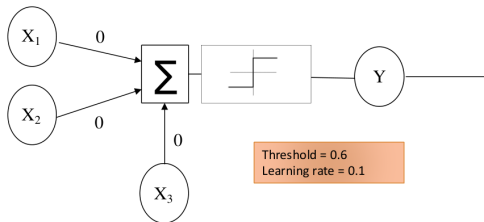


Perceptron Training – Epoch 5 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.1	-0.1	0.5	0.5	0	1	+0.1
0	1	1	-0.1	-0.1	0.6	0.5	0	1	+0.1
1	0	1							
1	1	1							

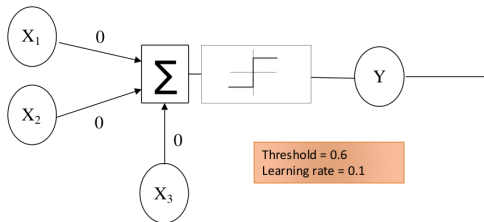


Perceptron Training – Epoch 5 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.1	-0.1	0.5	0.5	0	1	+0.1
0	1	1	-0.1	-0.1	0.6	0.5	0	1	+0.1
→ 1	0	1	-0.1	0.0	0.7	0.6	1	0	-0.0
1	1	1							

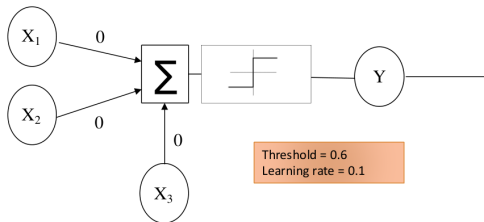


Perceptron Training – Epoch 5 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.1	-0.1	0.5	0.5	0	1	+0.1
0	1	1	-0.1	-0.1	0.6	0.5	0	1	+0.1
1	0	1	-0.1	0.0	0.7	0.6	1	0	∓ 0.0
1	1	1	-0.1	0.0	0.7	0.6	1	-1	-0.1



Perceptron Training – Convergence |

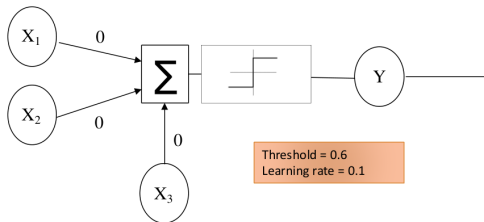


Perceptron Training – Epoch 6 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.1	0.6	0.6	1	0	∓ 0.0
0	1	1	-0.2	-0.1	0.6	0.5	0	1	+0.1
1	0	1	-0.2	0.0	0.7	0.5	0	1	+0.1
1	1	1	-0.1	0.0	0.8	0.7	1	-1	-0.1

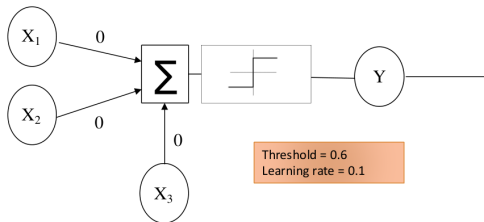


Perceptron Training – Epoch 6 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.1	0.7				
0	1	1							
1	0	1							
1	1	1							

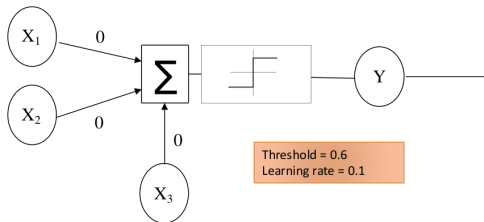


Perceptron Training – Epoch 6 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.1	0.7	0.7	1	0	∓ 0.0
0	1	1	-0.2	-0.1	0.7	0.6	1	0	∓ 0.0
1	0	1							
1	1	1							

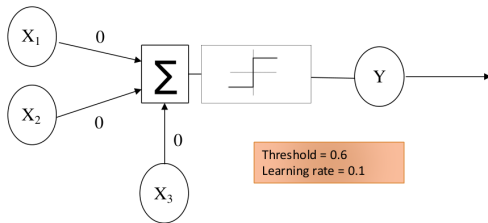


Perceptron Training – Epoch 6 (cont.) |

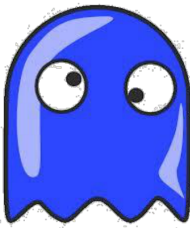
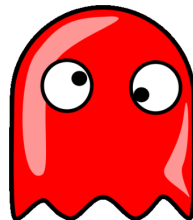
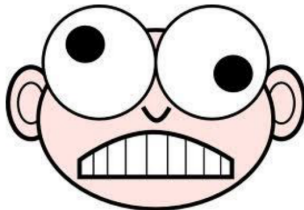
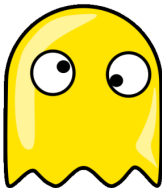
Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.1	0.7	0.7	1	0	∓ 0.0
0	1	1	-0.2	-0.1	0.7	0.6	1	0	∓ 0.0
1	0	1	-0.2	-0.1	0.7	0.5	0	1	+0.1
1	1	1	-0.1	-0.1	0.8	0.6	1	-1	-0.1



Perceptron Training – Convergence |

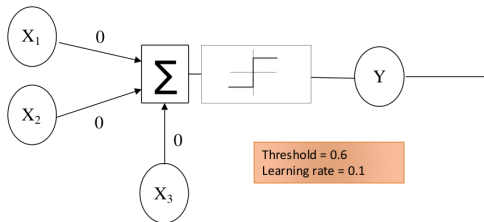


Perceptron Training – Epoch 7 |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.2	0.7	0.7	1	0	∓ 0.0
0	1	1	-0.2	-0.2	0.7	0.5	0	1	+0.1
1	0	1							
1	1	1							

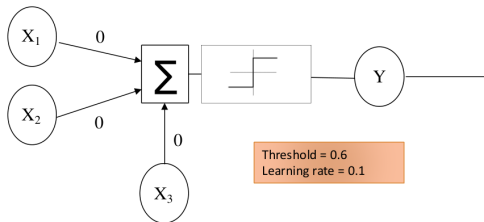


Perceptron Training – Epoch 7 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.2	0.7	0.7	1	0	∓ 0.0
0	1	1	-0.2	-0.2	0.7	0.5	0	1	+0.1
1	0	1	-0.2	-0.1	0.8	0.6	1	0	∓ 0.0
1	1	1	-0.2	-0.1	0.8	0.5	0	0	∓ 0.0

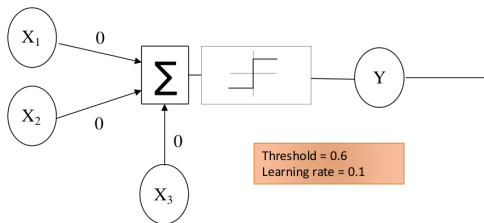


Perceptron Training – Epoch 7 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.1	0.8	0.8	1	0	∓ 0.0
0	1	1							
1	0	1							
1	1	1							

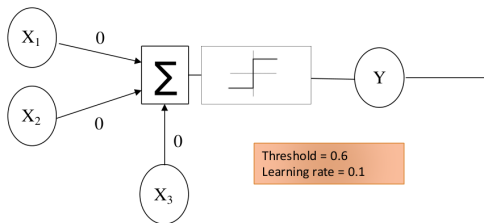


Perceptron Training – Epoch 7 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.1	0.8	0.8	1	0	∓ 0.0
0	1	1	-0.2	-0.1	0.8	0.7	1	0	∓ 0.0
1	0	1							
1	1	1							

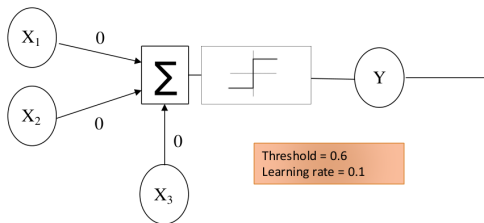


Perceptron Training – Epoch 7 (cont.) |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.1	0.8	0.7	1	0	∓ 0.0
0	1	1	-0.2	-0.1	0.8	0.7	1	0	∓ 0.0
→ 1	0	1	-0.2	-0.1	0.8	0.6	1	0	∓ 0.0
1	1	1							

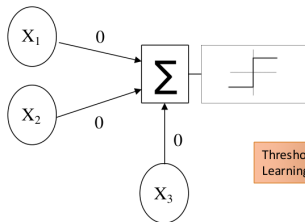


Perceptron Training – Converged! |

Training a Perceptron

Input 1	Input 2	Desired Output
0	0	1
0	1	1
1	0	1
1	1	0

X_1	X_2	X_3	W_1	W_2	W_3	Sum	Output	Error	Correction
0	0	1	-0.2	-0.1	0.8	0.7	1	0	∓ 0.0
0	1	1	-0.2	-0.1	0.8	0.7	1	0	∓ 0.0
1	0	1	-0.2	-0.1	0.8	0.6	1	0	∓ 0.0
1	1	1	-0.2	-0.1	0.8	0.5	0	0	∓ 0.0



Threshold = 0.6
Learning rate = 0.1



Question: How to Update the Weights? |

The Error State

- **Input Pattern:** $X_1 = 1$,
 $X_2 = 1$
- **Target (T):** 1
- **Threshold (θ):** 1.0
- **Learning Rate (LR):**
0.3

Initial Weights

- $w_1 = 0.5$
- $w_2 = -0.4$

The Learning Rule

1. Calculate Output (O):

$$Sum = (X_1 \times w_1) + (X_2 \times w_2)$$

If $Sum \geq \theta$, then $O = 1$, else $O = 0$

2. Calculate Error:

$$Err = T - O$$

3. Update Rule:

If $Err \neq 0$, then:

$$w_i = w_i + (LR \times X_i \times Err)$$

Question:

How do we apply this rule, and what are the w values after 1, 2 and 3 updates?

Iteration 1: Initial State & First Update |

System Parameters

- **Input:** $X_1 = 1, X_2 = 1$
- **Target (T):** 1
- **Threshold (θ):** 1.0
- **Learning Rate (LR):** 0.3

Initial Weights

- $w_1 = 0.5, w_2 = -0.4$

Update 1 Calculation

1. Evaluate Output:

$$Sum = (1 \times 0.5) + (1 \times -0.4) = 0.1$$

Since $0.1 < 1.0$, $O = 0$.

$$\text{Error} = 1 - 0 = 1.$$

2. Apply Learning Rule:

$$w_i = w_i + (0.3 \times X_i \times 1)$$

- $w_1 = 0.5 + 0.3 \rightarrow \mathbf{w_1 = 0.8}$

- $w_2 = -0.4 + 0.3 \rightarrow \mathbf{w_2 = -0.1}$

Iteration 2: Pushing Closer to the Threshold |

Current State: $w_1 = 0.8$, $w_2 = -0.1$, $\theta = 1.0$.

Input: $X_1 = 1$, $X_2 = 1$, Target $T = 1$.

Update 2 Calculation

1. Re-evaluate Output:

$$Sum = (1 \times 0.8) + (1 \times -0.1) = 0.8 - 0.1 = \mathbf{0.7}.$$

Since $0.7 < 1.0$, the network still outputs $O = 0$.

Error is still 1.

2. Apply Learning Rule:

Apply the exact same adjustment step (+0.3 to each weight):

- $w_1 = 0.8 + (0.3 \times 1 \times 1) \rightarrow \mathbf{w_1 = 1.1}$

- $w_2 = -0.1 + (0.3 \times 1 \times 1) \rightarrow \mathbf{w_2 = 0.2}$

Iteration 3: Reaching Convergence |

Current State: $w_1 = 1.1$, $w_2 = 0.2$, $\theta = 1.0$.

Input: $X_1 = 1$, $X_2 = 1$, Target $T = 1$.

Final Evaluation (Convergence)

1. Re-evaluate Output:

$$Sum = (1 \times 1.1) + (1 \times 0.2) = 1.1 + 0.2 = \mathbf{1.3}.$$

Because $\mathbf{1.3} \geq \mathbf{1.0}$ (our threshold θ), the network successfully fires!

Output $O = 1$.

2. Calculate Error:

$$Err = T - O = 1 - 1 = \mathbf{0}.$$

Result: Since the Error is 0, no further weight updates occur for this pattern. The network has successfully learned to output 1 for the (1, 1) input.

Evaluating Model Performance |

- **Beyond Convergence:** Just because a neural network converged during training doesn't mean it will perform well on unseen data.
- **Train vs. Test:** We must evaluate our models on a separate test dataset to measure their true generalization capability.
- For classification tasks (like predicting whether an image is a cat or a dog), calculating the overall error isn't enough. We need specialized metrics to understand exactly *how* the model is succeeding or failing.

The Confusion Matrix |

A **Confusion Matrix** is a table layout that visualizes the performance of a classification algorithm.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

- **TP**: The model correctly predicted the positive class.
- **TN**: The model correctly predicted the negative class.
- **FP**: The model incorrectly predicted positive (*Type I error*).
- **FN**: The model incorrectly predicted negative (*Type II error*).

Calculating Accuracy (Classification) |

The Accuracy Formula

For classification tasks, accuracy is simply the fraction of predictions our model got exactly right.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

Using the terms from the Confusion Matrix:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Important Limitation:

- Accuracy is only valid when predicting **categorical** outputs (e.g., Cat vs. Dog, Spam vs. Not Spam).
- We **cannot** use this exact formula for continuous outputs.

Evaluation Metrics: Accuracy |

Accuracy

The ratio of correctly predicted observations to the total observations.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Pros:** Highly intuitive and easy to explain.
- **Cons:** Can be extremely misleading when the dataset is **imbalanced**.

Example: If 99% of emails are normal and 1% are spam, a model that lazily guesses "normal" every single time achieves 99% accuracy, but it is completely useless at detecting spam!

Evaluation Metrics: Precision & Recall |

Precision

Out of all the cases the model *predicted* as positive, how many were actually positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Focuses on minimizing False Positives.

Recall (Sensitivity)

Out of all the *actual* positive cases, how many did the model correctly find?

$$\text{Recall} = \frac{TP}{TP + FN}$$

Focuses on minimizing False Negatives.

The Trade-off: There is usually an inverse relationship between Precision and Recall. Tuning a network to increase one often decreases the other.

Evaluation Metrics: F1 Score |

F1 Score

The harmonic mean of Precision and Recall. It provides a single metric that balances both concerns.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Excellent for evaluating models trained on highly imbalanced datasets.
- A high F1 score indicates that the model has both low false positives *and* low false negatives.
- Unlike the arithmetic mean, the harmonic mean heavily punishes extreme values. If either Precision or Recall drops to near 0, the F1 Score will also collapse.

Evaluating Regression Models |

- When a neural network outputs a **continuous numerical value** (Regression), the concept of "True Positives" or counting "Exact Matches" no longer applies.
- Instead of asking *"Did the model guess the exact right category?"*, we must ask *"How close is the prediction to the actual value?"*
- We evaluate regression models by measuring the **Error** (or Residual). This is the distance between the predicted value ($\hat{y}$) and the actual target value (y).

Regression Metrics: MSE & RMSE |

Mean Squared Error (MSE)

Measures the average of the squares of the errors. Because the errors are squared, it heavily penalizes large errors (outliers).

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE)

The square root of MSE. It brings the error metric back to the original units of the target variable, making it much easier for humans to interpret.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Regression Metrics: MAE & R^2 |

Mean Absolute Error (MAE)

The average of the absolute differences between predictions and actual values.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

It treats all errors equally and is less sensitive to extreme outliers than MSE.

R-Squared (R^2 Score)

Measures the proportion of variance in the target variable that is explained by the model.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

This acts as the closest equivalent to "accuracy" for regression. An R^2 of 1.0 is perfect, while 0.0 is no better than guessing the average.